

# MECHA CONNECT

## The Engineering Handbook

*AI-Powered Roadside Assistance, from Architecture to Release*

Frontend Lock Candidate · Version 1.0.0+1  
Flutter 3.29.2 · Dart ^3.7.2 · Provider Architecture  
[github.com/Jagadeeshrelangi/mc\\_repo](https://github.com/Jagadeeshrelangi/mc_repo) · branch main

Documentation Build v2.2 · Cleanup & Freeze Sprint · 5 August 2026

# Table of Contents

## Part I — Product & Market

1. Introduction & Vision
2. Problem & Opportunity
3. Business Model & Unit Economics
4. Target Audience & Personas
5. Competitive Landscape
6. Feature Overview & Roadmap

## Part II — Design & Experience

7. User Experience & Journeys
8. UI/UX Design System
9. Technology Stack

## Part III — Engineering Architecture

10. Frontend Architecture
11. Feature Modules Deep Dive
12. Navigation & App Shell
13. Design Tokens & Responsive System
14. Data Layer & Database Blueprint
15. API Contract & Integration

## Part IV — Quality, Release & Operations

16. Testing & QA
17. Release History & Versioning
18. Certification & Frontend Lock
19. Deployment & DevOps

## Part V — Risk & Roadmap

20. Risks & Mitigations
21. Future Scope & Next Steps

## Appendices

- A. Glossary
- B. ID Scheme Quick Reference
- C. Figure Index
- D. Source Document Index

## Revision & Scope

### Revision History

| Version | Date       | Milestone                             |
|---------|------------|---------------------------------------|
| 0.0.1   | 2026-07-20 | Project initialised                   |
| 0.1.0   | —          | Sprints 1.1–1.3 (foundations)         |
| 0.4.0   | —          | Sprint 1.6 — Mechanic booking         |
| 1.2.0   | 2026-07-30 | Sprint 1.7A — Fuel delivery           |
| 1.9.0   | 2026-08-02 | Sprint 1.9B — RC1 certification       |
| 1.9.1   | 2026-08-05 | Sprint 1.9b — Close                   |
| 1.9.2   | 2026-08-05 | Sprint 1.9b — Final review            |
| 1.0.0+1 | 2026-08-05 | RC1 release — Frontend Lock Candidate |

*Intermediate dates between 0.1.0 and 1.2.0 are not recorded in the source bundle and are left blank rather than estimated.*

### How This Handbook Was Built

This handbook is compiled entirely from the Mecha Connect documentation bundle — the merged knowledge base, machine-readable project data, module and workflow files, database and API contracts, navigation maps, and the fifteen architecture diagrams that accompany them. It does not draw on outside knowledge of the Mecha Connect codebase, and it does not fill gaps with invented detail.

#### **FROZEN (RC1)**

Where the bundle marks something as frozen for Release Candidate 1, this handbook preserves that wording exactly. Where the bundle is silent on a topic — competitive landscape and deployment operations, most notably — this handbook says so plainly and labels the section Future Work rather than guessing.

### Who This Handbook Is For

Investors and technical reviewers evaluating engineering maturity ahead of Sprint 2; engineering managers planning the backend migration; senior developers who need the architecture's shape before touching code; and new contributors who need one document that explains what the app does, how it is built, and what is still ahead.

# Chapter 1 — Introduction & Vision

*What Mecha Connect is, who it serves, and the idea that holds its five feature modules together.*

## 1.1 The Product

Mecha Connect is an AI-powered roadside assistance ecosystem for Indian vehicle owners, built as a single Flutter application that connects owners, mechanics, fuel partners, and spare-part sellers. The team's own shorthand for it is direct: "Uber + Swiggy + AI Assistant" for vehicle services — on-demand help, on-demand delivery, and an AI layer that tries to understand the problem before a human ever gets involved.

## 1.2 Mission

The mission stated in the product documentation is to give vehicle owners a single place to book a verified mechanic, order fuel delivery, buy spare parts, and get an AI diagnosis — collapsing what today is four separate, often informal transactions into one trusted app.

## 1.3 Unique Selling Points

- AI-powered diagnosis before booking — symptoms go in, a probable fault and a cost estimate come out, before the user commits to anything.
- End-to-end booking — diagnosis flows straight into payment, without the user re-explaining the problem to a second app or a phone call.
- A multi-service ecosystem — Mechanic, Fuel, and Parts under one roof, sharing one order history and one AI assistant.
- Real-time tracking of help arriving, the way ride-hailing and food-delivery apps have trained users to expect.
- Verified professionals with transparent, upfront pricing — the two things informal roadside repair in India is usually missing.

## 1.4 Where the Project Stands Today

At the time of writing, Mecha Connect is at Release Candidate 1 (RC1), version 1.0.0+1, and its frontend has been declared a Frontend Lock Candidate — frozen pending approval, with a verification gate of zero static-analysis issues and 162 of 162 automated tests passing. The entire data layer is currently in-memory mock repositories with simulated network latency and deliberate failure injection, standing in for a backend that does not yet exist in production. That distinction — a fully realised frontend sitting on a deliberately fake backend — is the single most important fact to hold onto while reading the rest of this handbook, because it shapes almost every architectural decision described in Part II.

### FROZEN (RC1) — Terminology

This handbook uses the phrase "Frontend Lock Candidate" exactly as the source documentation specifies, and never the phrase "RC1 Certified," which the project has explicitly retired.

## Key Takeaways

- Mecha Connect unifies mechanic booking, fuel delivery, spare-parts marketplace, and AI diagnosis in one Flutter app.
- RC1 (v1.0.0+1) is a Frontend Lock Candidate: the UI is frozen and verified, but every data source today is an in-memory mock.
- The gap between a real frontend and a mock backend is the central engineering story of the next two sprints.

## References

- *Mecha Connect Master Project Knowledge Base, §1–2*
- *MASTER\_PROJECT\_DATA.json*

## Related Chapters

[Chapter 3 — Business Model](#) · [Chapter 10 — Frontend Architecture](#) · [Chapter 18 — Certification & Frontend Lock](#)

## Chapter 2 — Problem & Opportunity

*The gap Mecha Connect is built to close, and where the source documentation runs out.*

### 2.1 The Problem, As Documented

The product vision (Chapter 1) implies the problem it is solving even where the bundle does not spell it out as a standalone problem statement: vehicle owners today coordinate mechanics, fuel, and spare parts through separate, often informal and untracked channels, with no shared history, no verified pricing, and no diagnostic step before a repair begins. Mecha Connect's answer is to fold all three into one app with one AI assistant sitting in front of them.

### 2.2 Opportunity Signals in the Bundle

The business model (Chapter 3) and the three personas (Chapter 4) are the strongest evidence the source material offers for market opportunity — a commuter who needs a mechanic in under ten minutes, a night-time safety-conscious rider, and a small fleet operator who needs bulk booking and cost tracking are three distinct, plausible demand signals for the same underlying product.

#### **FUTURE WORK — NOT AVAILABLE YET — Market Sizing & Opportunity Analysis**

A dedicated problem statement — market size, total addressable market, growth trend, and a named competitive gap — is referenced by the documentation's own table of sources (as feeding Chapter 5) but is not present anywhere in the delivered bundle. It is left here as Future Work rather than inferred from the personas alone.

### Key Takeaways

- The problem is implied clearly by the product vision and personas, but no standalone market-sizing document exists in the source bundle.
- Three personas provide qualitative evidence of demand; no quantitative opportunity sizing is available yet.

### References

- *Mecha Connect Master Project Knowledge Base, §2.1–2.2*

### Related Chapters

[Chapter 4 — Target Audience & Personas](#) · [Chapter 5 — Competitive Landscape](#)

## Chapter 3 — Business Model & Unit Economics

*How Mecha Connect intends to make money, and what the early unit economics look like.*

### 3.1 Revenue Streams

| Stream                           | Structure                        |
|----------------------------------|----------------------------------|
| Booking commission               | 15–20% of mechanic booking value |
| Fuel delivery margin             | ₹20–50 per delivery              |
| Seller listing fee               | ₹99 / month, spare-parts sellers |
| Featured mechanic placement      | ₹499 / month                     |
| AI diagnosis API (B2B, Sprint 3) | ₹999 per fleet per month         |

### 3.2 Unit Economics

| Metric                          | Value          |
|---------------------------------|----------------|
| Customer Acquisition Cost (CAC) | ₹150           |
| Average Order Value (AOV)       | ₹450           |
| Lifetime Value (LTV)            | ₹2,250         |
| Payback period                  | 3 transactions |

#### Reading the Numbers

An LTV:CAC ratio of 15:1 and a three-transaction payback period are, on paper, strong early-stage economics — but the bundle does not state the cohort, time window, or assumptions (repeat-purchase rate, churn) behind these figures, so they should be read as directional targets rather than audited results.

#### FUTURE WORK — NOT AVAILABLE YET — Monetization Timeline & Cohort Detail

A monetisation timeline table is referenced in the source material's own index but its contents — which revenue streams activate in which sprint, and the assumptions behind CAC/AOV/LTV — are not present in this bundle.

#### Key Takeaways

- Five revenue streams span commission, delivery margin, listing fees, featured placement, and a B2B AI API planned for Sprint 3.
- Early unit economics (CAC ₹150, LTV ₹2,250, 3-transaction payback) look strong but are not shown with supporting cohort data.

#### References

- *Mecha Connect Master Project Knowledge Base, §2.4*

#### Related Chapters

[Chapter 4 — Target Audience & Personas](#) · [Chapter 21 — Future Scope & Next Steps](#)

## Chapter 4 — Target Audience & Personas

*Three named personas anchor Mecha Connect's product decisions, from booking speed to safety features to fleet tools.*

### 4.1 Persona Summary

| Persona | Profile                                        | Core need                                                   |
|---------|------------------------------------------------|-------------------------------------------------------------|
| Rajesh  | 28, daily commuter, Hyderabad, rides an Activa | A verified mechanic in under 10 minutes                     |
| Priya   | 32, Bangalore, drives an i10                   | Night-time safety — tracked arrival, SOS, verified profiles |
| Vikram  | 40, Chennai, runs 10 delivery bikes            | Bulk booking, cost tracking, ratings across a fleet         |

### 4.2 What the Personas Drive in the Product

- Rajesh's speed requirement is reflected in the nearby-mechanic sorting and live-tracking features in the Mechanic module (Chapter 11).
- Priya's safety requirement maps directly to verified-profile display, live tracking, and the SOS entry point on the Home dashboard (Chapter 7).
- Vikram's fleet-operator needs point toward the B2B AI-diagnosis API planned for Sprint 3 (Chapter 3) and the multi-vehicle support already present in the Profile module (Chapter 11).

#### Key Takeaways

- Three personas — a fast-need commuter, a safety-conscious driver, and a fleet operator — cover distinct segments with one shared product.
- Each persona's stated need traces to a specific, already-built product feature rather than remaining aspirational.

#### References

- *Mecha Connect Master Project Knowledge Base, §2.2*

#### Related Chapters

[Chapter 7 — User Experience & Journeys](#) · [Chapter 11 — Feature Modules Deep Dive](#)



## Chapter 5 — Competitive Landscape

*A placeholder chapter, deliberately left incomplete rather than fabricated.*

### FUTURE WORK — NOT AVAILABLE YET

The source bundle's own reference index (manifests/tables\_manifest.md) points to a competitive-analysis table sourced from the product requirements document, but that document is not included anywhere in the delivered bundle.

No competitor names, positioning comparisons, or differentiation analysis exist in any file reviewed for this handbook.

Rather than infer competitors from general knowledge of the Indian roadside-assistance or on-demand-services market — which this handbook's brief explicitly forbids — this chapter is left as Future Work.

### 5.1 What Would Complete This Chapter

- A named list of direct and adjacent competitors (roadside assistance, fuel delivery, spare-parts marketplaces, and AI diagnostic tools, operating in India).
- A feature-by-feature or positioning comparison table.
- An explicit statement of Mecha Connect's differentiation beyond the USP list already captured in Chapter 1.

### Key Takeaways

- This chapter is intentionally incomplete: no competitive-landscape source material exists in the bundle reviewed.
- Completing it requires the product requirements document referenced by, but absent from, the current documentation set.

### References

- *manifests/tables\_manifest.md (source citation only; underlying document not delivered)*

### Related Chapters

[Chapter 1 — Introduction & Vision](#) · [Chapter 3 — Business Model & Unit Economics](#)

## Chapter 6 — Feature Overview & Roadmap

*What has shipped, what the original plan said, and where the two disagree.*

### 6.1 What's Actually Built at RC1

The frontend at RC1 covers a wider surface than the original sprint plan promised. Splash, onboarding, authentication, the Home dashboard, the full mechanic-booking lifecycle, AI diagnosis and chat, fuel delivery, the parts marketplace, the full Profile suite (wallet, rewards, vehicles, addresses), dark mode, a responsive layout, and the five-tab shell are all shipped in the frontend — as mocked, fully interactive experiences.

#### ENGINEERING NOTE — Docs vs. Code

The product requirements document marks Fuel Delivery and Marketplace as "Sprint 2" scope. Both shipped in the RC1 frontend anyway, running against mock repositories. This is a genuine, documented gap between the original plan and what was actually built — and it is a gap in the team's favour: the frontend is ahead of its own roadmap.

### 6.2 Feature Inventory by Module

| Module        | Screens | Status at RC1                             |
|---------------|---------|-------------------------------------------|
| Home          | 2       | Shipped — dashboard + search              |
| Auth          | 3       | Shipped — local session only              |
| Mechanic      | 11      | Shipped — full booking lifecycle          |
| AI Assistant  | 5       | Shipped — mock keyword engine             |
| Fuel Delivery | 8       | Shipped — originally planned for Sprint 2 |
| Marketplace   | 8       | Shipped — originally planned for Sprint 2 |
| Profile       | 12      | Shipped — full account centre             |

### 6.3 Roadmap

| Sprint   | Focus                                                                                                       |
|----------|-------------------------------------------------------------------------------------------------------------|
| Sprint 2 | FastAPI deployment, PostgreSQL migration, full API contract implementation, Firebase Auth, Maps integration |
| Sprint 3 | Production polish; B2B AI-diagnosis API monetisation begins                                                 |
| Sprint 4 | Partner app (for mechanics / fuel partners)                                                                 |
| Sprint 5 | Admin dashboard                                                                                             |

#### FUTURE WORK — NOT AVAILABLE YET — Roadmap Detail

Sprints 3, 4, and 5 are named in the source knowledge base but not broken down into features, timelines, or acceptance criteria. That detail is Future Work.

### Key Takeaways

- RC1's frontend already covers Fuel Delivery and Marketplace, features the original roadmap deferred to Sprint 2.
- Sprint 2 is backend-focused: FastAPI, PostgreSQL, the full API contract, Firebase Auth, and Maps.
- Sprints 3–5 are named (polish, partner app, admin dashboard) but not yet detailed.

## References

- *Mecha Connect Master Project Knowledge Base, §2.5, §9*
- *MASTER\_PROJECT\_DATA.json*

## Related Chapters

[Chapter 9 — Technology Stack](#) · [Chapter 15 — API Contract & Integration](#) · [Chapter 21 — Future Scope & Next Steps](#)

## Chapter 7 — User Experience & Journeys

---

*Seven end-to-end journeys, each grounded in the product's Mermaid workflow diagrams and their documented failure and recovery paths.*

### 7.1 Auth & Session

The splash screen reads a local `is_logged_in` flag and, after a 450-millisecond fade, routes the user to the five-tab shell, the login screen, or onboarding — whichever applies. Login, sign-up, and forgot-password are validated locally (email format, a six-character password minimum, confirm-match) with no credentials stored on device or server at RC1. Google Sign-In is specified but not yet wired. Logout is the only reverse path, clearing the stack back to the login screen.

### 7.2 Mechanic Booking

A user opens the Mechanic module through the Services tab, browses featured and nearby mechanics, opens a detail screen with ratings, services, and working hours, and can optionally run the AI module's diagnosis service against a vehicle form before booking — a genuine cross-module handoff that costs nothing over the network, because it runs against the same mock AI repository the AI tab itself uses. Booking confirmation clears the navigation stack; live tracking follows a mechanic marker on a three-second timer that never rebuilds the underlying map; completion leads into rating and review, and the booking lands in booking history.

### 7.3 Fuel Delivery

Fuel delivery leans hardest on location services of any module: GPS acquisition is deliberately capped at a ten-second settings limit and a twelve-second hard timeout, with a manual "Set Manually" fallback to a default Bengaluru coordinate — a fix, per the source documentation, for a blank-screen bug on GPS-less devices discovered in an earlier sprint. From there the flow is familiar: pick a saved vehicle, a fuel type, a nearby station, and a quantity; price is computed as fuel cost plus delivery charge plus platform fee plus taxes; payment leads to live tracking through a defined status sequence, and completion generates an invoice.

### 7.4 Marketplace Checkout

Browsing surfaces featured, best-seller, trending, flash-deal, and recommended flags across a forty-product catalogue. Product cards use a scoped rebuild (`context.select`) so that toggling a wishlist heart does not re-render the whole card. Checkout applies a coupon, an address, and a payment method, then places the order — which does two things at once: it returns a typed order object for the confirmation screen, and it writes an entry into the shared Orders tab, the single feed that Chapter 14 covers in detail.

### 7.5 AI Diagnosis & Assistant

The AI tab opens onto five seeded conversations (two pinned), suggested questions, and quick actions. Chat replies come from a keyword knowledge base rather than a generative model at RC1. Guided diagnosis takes a vehicle, a problem, and a set of symptoms and returns a structured payload — probable causes, severity, an estimated cost, a recommended action, and a confidence score. Action buttons deep-

link straight into Mechanic, Fuel, or Marketplace, making the AI tab a genuine dispatcher for the rest of the app rather than an isolated chatbot.

## 7.6 Profile & Vehicle Management

The Profile tab is the account centre: profile details, saved vehicles and addresses (both sorted default-first, a frozen ordering rule), wallet and rewards, unified order history (read from the same shared list Marketplace writes to), notification settings, privacy, support, and the theme picker — which is a dialog, not a separate route.

## 7.7 Orders Tab — the Unified Feed

The Orders tab is deliberately thin: it renders a single shared list, grouped by category, that Marketplace checkout writes into and Profile order history reads from. Because the tab lives inside an IndexedStack that keeps all five tabs mounted, it only rebuilds when the list itself notifies — not on every frame — which is both a performance decision and the reason Profile and Orders never disagree with each other about what's been ordered.



Figure 1. The Orders tab as a single shared feed written by Marketplace and read by Profile.

### Key Takeaways

- Every major journey funnels through the same repository-and-provider pattern, which is why the flows read consistently across five very different feature areas.
- Two cross-module seams matter most: AI diagnosis reused inside Mechanic booking, and the single Orders feed shared by Marketplace and Profile.
- Failure and recovery paths (GPS denial, network exceptions, out-of-stock, mechanic unavailability) are designed in at RC1, not deferred to Sprint 2.

### References

- `workflows/*.md` (seven workflow files)
- `modules/overview.md` — Cross-Module Edges

### Related Chapters

[Chapter 11 — Feature Modules Deep Dive](#) · [Chapter 14 — Data Layer & Database Blueprint](#) · [Chapter 20 — Risks & Mitigations](#)

## Chapter 8 — UI/UX Design System

*The tokens, palette, and interaction rules that give Mecha Connect a consistent look across fifty screens.*

### 8.1 Brand Palette

| Token        | Value                                 | Use                               |
|--------------|---------------------------------------|-----------------------------------|
| Brand Orange | #F15A22 (family; gradient to #D44A15) | Primary brand colour              |
| Brand Blue   | #4285F4 (family)                      | Secondary / informational accents |
| Success      | #10B981                               | Confirmations, positive states    |
| Error        | #EF4444                               | Failures, destructive actions     |
| Warning      | #F59E0B                               | Caution states                    |
| Info         | #3B82F6                               | Informational callouts            |

#### Dark Mode

Dark mode is treated as a premium palette in its own right rather than a simple inversion of the light theme — background #0E1117, card #1A1D24, primary accent shifted to #FF6A2A for contrast against the darker surface.

### 8.2 Spacing, Radius, Elevation

- Spacing scale: a 4px base unit running from xxs (2px) up to hero (64px).
- Corner radii: 6 / 10 / 14 / 18 / 22 / 28 / full.
- Elevation: six levels, 0 through 5.

### 8.3 Typography

Display headings use Space Grotesk; body text uses the platform default. The type scale is responsive, clamped between 0.88x and 1.2x so text stays legible on both small phones and tablets without ever ballooning out of proportion.

### 8.4 Responsive Rules

Two breakpoints govern layout: 600px (mobile) and 1024px (tablet). A ConstrainedContent wrapper caps content width at 480px on larger screens, keeping the mobile-first design from stretching awkwardly wide.

### 8.5 Interaction Patterns

- Pill-shaped buttons and rounded cards throughout.
- Skeleton loaders rather than spinners for content-shaped loading states.
- Branded illustrations for both error and empty states, rather than generic system dialogs.
- The bottom navigation bar's active tab uses the accent colour at 8% background opacity.
- Interactive controls carry explicit Semantics labels for screen-reader accessibility.

**TIP — Design System as an Engineering Artifact**

Because every token above is enumerable and frozen, a new screen can be styled correctly without ever opening a design tool — which is exactly what let the frontend ship fifty screens with a genuinely consistent look.

**Key Takeaways**

- The design system is fully tokenised: colour, spacing, radius, elevation, and typography are all enumerable values, not ad hoc choices.
- Dark mode is a deliberately designed second palette, not an automatic inversion.
- Accessibility (Semantics labelling, reduced-motion handling) is built into the token system rather than bolted on afterward.

**References**

- *UI\_DESIGN\_SYSTEM.md* (via Mecha Connect Master Project Knowledge Base, §7)

**Related Chapters**

[Chapter 11 — Feature Modules Deep Dive](#) · [Chapter 13 — Design Tokens & Responsive System](#)

## Chapter 9 — Technology Stack

*What the client is built with today, and what the backend is built with once Sprint 2 lands.*

### 9.1 Client Stack

| Layer             | Choice         |
|-------------------|----------------|
| Framework         | Flutter 3.29.2 |
| Language          | Dart ^3.7.2    |
| State management  | Provider       |
| Bottom navigation | google_nav_bar |
| Maps              | flutter_map    |
| Location          | geolocator     |

#### Key Client Dependencies

provider 6.1.5 · google\_nav\_bar 5.0.7 · device\_preview 1.2.0 · shared\_preferences 2.5.3 · flutter\_dotenv 5.2.1 · latlong2 0.9.1 · geolocator 13.0.4 · permission\_handler 11.4.0 · flutter\_map 7.0.2 · http 1.6.0 · flutter\_lints 5.0.0.

### 9.2 Backend Stack (Sprint 2 Target)

| Layer          | Choice                                                           |
|----------------|------------------------------------------------------------------|
| API framework  | FastAPI                                                          |
| Database       | PostgreSQL 15                                                    |
| Cache          | Redis                                                            |
| Authentication | Firebase Auth + JWT                                              |
| AI / ML        | Gemini (chat), FAISS (retrieval), XGBoost (fault classification) |

#### ENGINEERING NOTE — Scaffold, Not Shipped

The backend stack above describes target architecture. A partial scaffold already exists for the AI surface (conversation, diagnosis, and knowledge endpoints, backed by a chat service, a diagnosis service, and a retrieval-augmented-generation service over a FAISS index). Marketplace, mechanic, fuel, and profile endpoints are not yet scaffolded. See Chapter 15 for the full contract.

### 9.3 Environment

| Item                   | Value                                            |
|------------------------|--------------------------------------------------|
| Repository             | github.com/Jagadeeshrelangi/mc_repo, branch main |
| Backend version string | 1.0.0                                            |
| Python                 | 3.13                                             |
| Client package manager | pub                                              |



## Key Takeaways

- The client stack is fixed and versioned precisely; the backend stack is a named target, partially scaffolded for AI only.
- Flutter's Provider package, not Riverpod or Bloc, is the chosen state-management approach — see Chapter 10 for how it's wired.
- AI/ML choices (Gemini, FAISS, XGBoost) are already reflected in a working backend scaffold, ahead of the rest of the API surface.

## References

- *metadata/version\_matrix.md*
- *MASTER\_PROJECT\_DATA.json — backend*

## Related Chapters

[Chapter 10 — Frontend Architecture](#) · [Chapter 15 — API Contract & Integration](#)

## Chapter 10 — Frontend Architecture

*The layering rule that every module obeys, how the app boots, and the provider graph that wires it all together.*

### 10.1 The Layer Rule

One rule governs the entire frontend, and it is stated as frozen: Screens call Providers, Providers (each a ChangeNotifier) call Repositories, and Repositories are the only data source in the application. Screens never call HTTP directly, not even to a mock endpoint. This matters because it defines exactly what Sprint 2 has to change and exactly what it must not: repository internals swap from in-memory mock engines to FastAPI HTTP clients, while UI code and provider method signatures stay untouched.



Figure 2. The frozen frontend layering: Screens → Providers → Repositories → mock engines today, FastAPI clients from Sprint 2.

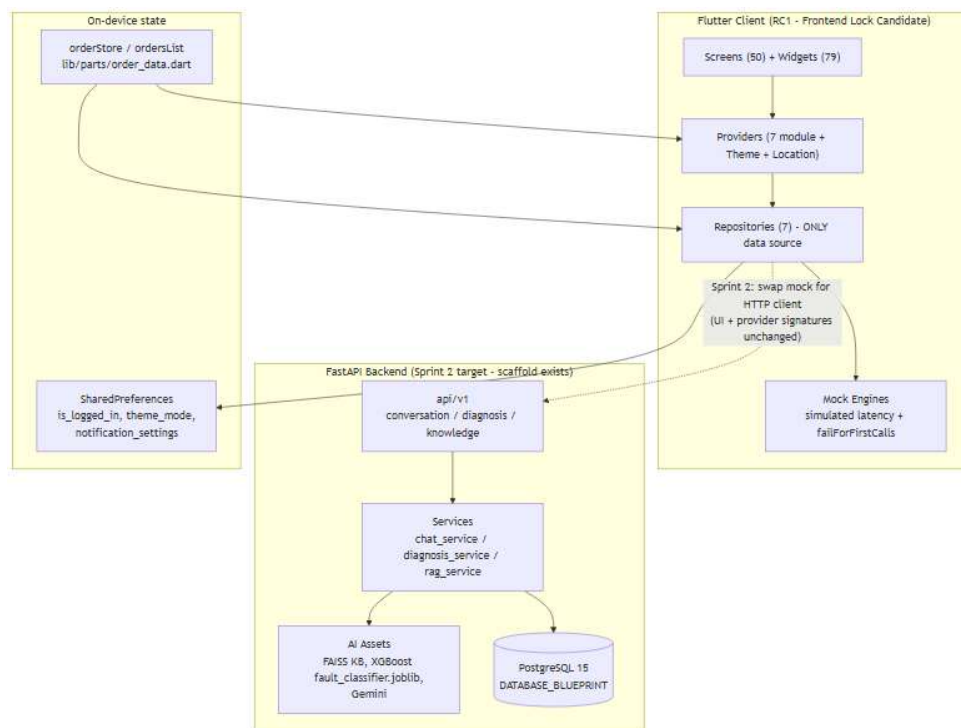


Figure 3. System architecture — client, backend scaffold, and target data stores.

### 10.2 App Entry

Startup follows a fixed sequence: `main()` loads environment variables on a best-effort basis, builds the root provider tree, and calls `runApp`. `MyApp` wraps everything in `DevicePreview` (active only in debug builds) and a `MaterialApp` configured with light, dark, and system theme modes, with a single named route — the root path, which resolves to the splash screen. Splash makes its routing decision after a 450-millisecond fade transition, and no development-only flags remain in this path.

### 10.3 The Provider Graph

Nine providers are constructed at the root of the app, in a fixed order, in `app_wiring.dart` — the single source of truth for how the app is wired together.

| # | Provider            | Constructed with                                                     |
|---|---------------------|----------------------------------------------------------------------|
| 1 | ThemeProvider       | ChangeNotifierProvider(create)                                       |
| 2 | LocationProvider    | Injectable, or a sensible default                                    |
| 3 | AuthProvider        | AuthService wrapping AuthRepository                                  |
| 4 | HomeProvider        | HomeRepository                                                       |
| 5 | MechanicProvider    | Default, repository injectable                                       |
| 6 | AiProvider          | Owns one AiRepository, shared with AiService and DiagnosisService    |
| 7 | ProfileProvider     | ProfileRepository over a SharedPreferences-backed notification store |
| 8 | FuelProvider        | Constructed with a dependency on LocationProvider                    |
| 9 | MarketplaceProvider | Injectable, or a sensible default                                    |

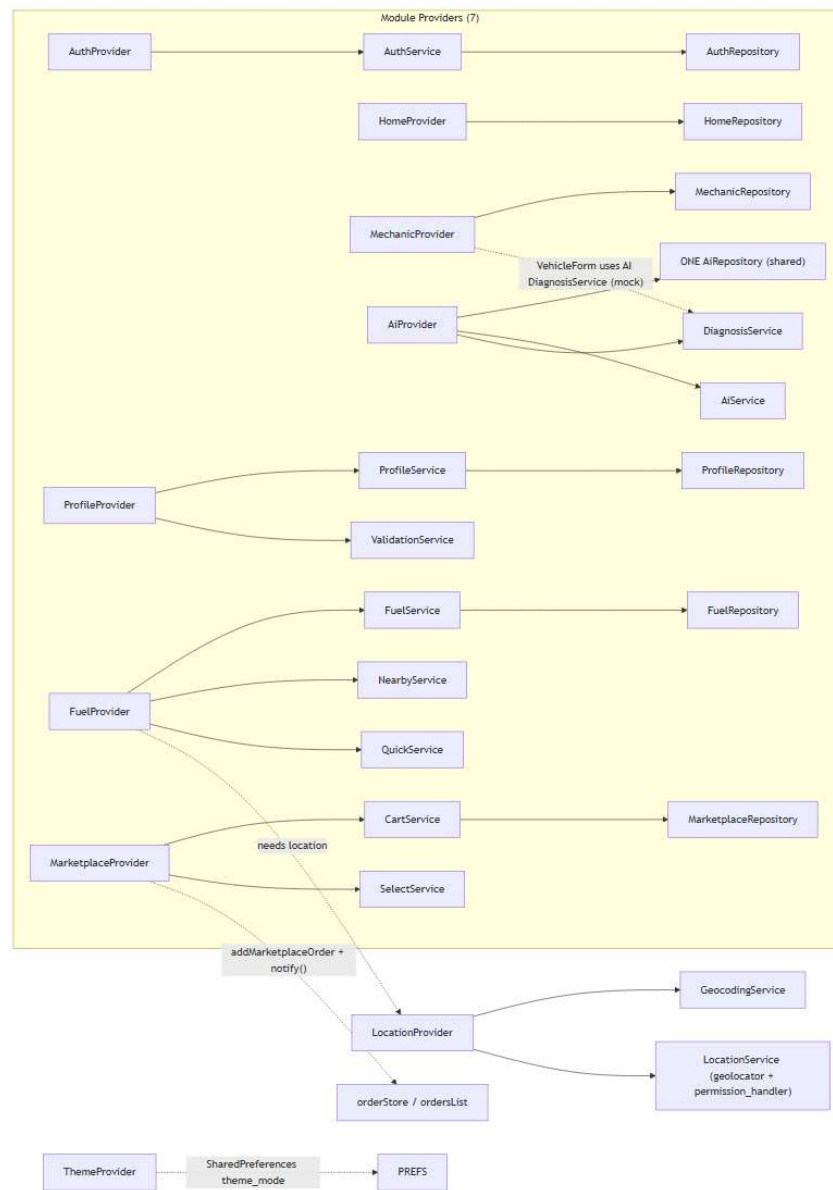


Figure 4. The nine root providers and their construction dependencies.

#### FROZEN (RC1) — Why Provider #6 Matters More Than It Looks

AiProvider owns exactly one AiRepository instance, and that same instance is shared with AIService and DiagnosisService. An earlier build had this wrong — three separate repository instances instead of one — which the team calls out explicitly as a fixed bug (the "triple-repo bug") in the workflow documentation. It's a good example of why the provider graph is treated as frozen: subtle sharing bugs like this are exactly what re-wiring risks reintroducing.

#### Injection Points for Testing

Four providers — Location, Fuel, Marketplace, and Profile — accept injected repositories, which is how the 162-test suite exercises real provider logic against controllable mock data, including deliberate failure injection, without ever touching the network.

#### Shared Singletons Outside the Provider Tree

Two objects live outside the provider graph entirely: `orderStore`, a `ChangeNotifier`, and `ordersList`, the list it guards — both defined in `lib/parts/order_data.dart`, and both are what makes the Orders tab and Profile order history a single source of truth rather than two lists that can drift apart.

## 10.4 The Five-Tab Shell

The app's body is an `IndexedStack`, which keeps all five tabs mounted in memory at once rather than tearing down and rebuilding screens on every tab switch — a deliberate trade of memory for responsiveness and state preservation, paired with a `google_nav_bar` navigation bar.

| Tab          | Screen                 |
|--------------|------------------------|
| 0 — Home     | HomeDashboard          |
| 1 — Services | ServiceSelectionScreen |
| 2 — Orders   | Orderscreen            |
| 3 — AI       | AiHomeScreen           |
| 4 — Profile  | ProfileScreen          |

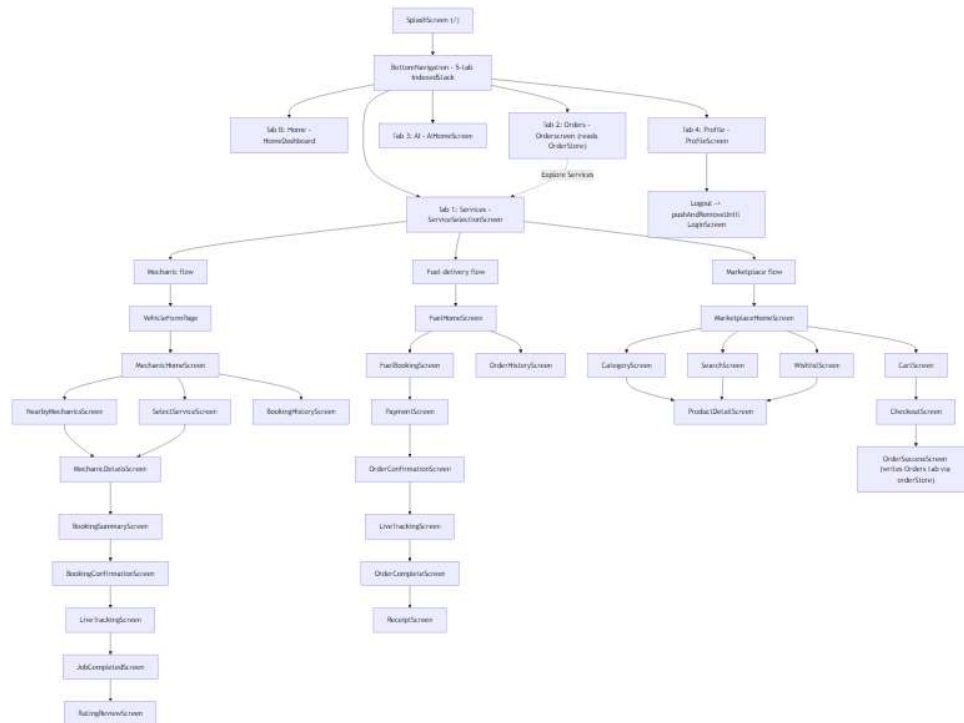


Figure 5. The five-tab shell, backed by an `IndexedStack`.

## 10.5 Repository Layer

Every feature module owns exactly one repository, and every repository simulates realistic network behaviour: a fixed latency (ranging from 700 to 900 milliseconds depending on module) and a `failForFirstCalls` mechanism that deterministically throws a typed exception for the first N calls, so that loading states, error states, and retry flows are all genuinely exercised in tests rather than assumed to work.



Figure 6. The repository seam: today's mock engines sit exactly where Sprint 2's FastAPI clients will sit tomorrow.

## 10.6 Folder Structure

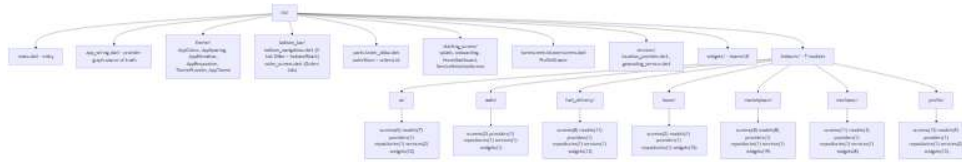


Figure 7. Feature-first folder organisation under `lib/features/`.

Modules are organised feature-first under `lib/features/`, each with its own models, providers, repositories, services, screens, and widgets — with one documented exception: the Home dashboard and Service Selection screen currently live under a legacy `lib/starting_screen/` path rather than `lib/features/home/`, a naming inconsistency the project itself tracks as technical debt (see Chapter 20).

## 10.7 ID Schemes

Every entity in the app carries a frozen, prefix-based identifier scheme, so that IDs are self-describing across logs, tests, and API payloads without needing a lookup table.

| Domain                                | Prefix             | Example            |
|---------------------------------------|--------------------|--------------------|
| Marketplace order                     | MKP-<year>-<0000>  | MKP-2026-0001      |
| Marketplace product                   | p-                 | p-spark-plug       |
| Coupon / offer                        | coupon- / offer-   | coupon-mecha10     |
| Shared orders feed                    | ORD-               | ORD-1001           |
| Fuel order                            | FUEL-<year>-<0000> | FUEL-2026-0009     |
| Fuel invoice                          | INV-<orderId>      | INV-FUEL-2026-0009 |
| Mechanic / service                    | m / svc_           | m1, svc_1          |
| AI conversation / message / diagnosis | ai- / m- / diag-   | ai-0001, diag-1    |
| Vehicle / address (counter from 200)  | veh- / addr-       | veh-101, addr-202  |
| Wallet transaction / reward / payment | txn- / rew- / pay- | txn-101            |

## 10.8 Frozen Scope at RC1

The shell and navigation, the design-system tokens, the feature modules, the data models, and the repository interfaces are all frozen for RC1. The permitted category of change is narrow and explicit: bug fixes, accessibility fixes, responsive-layout fixes, state-management fixes, and documentation. Any

change in this window must still pass the verification gate — zero analyzer issues and all 162 tests green — before it counts as done.

### Key Takeaways

- One layering rule — Screens → Providers → Repositories — is applied without exception across all seven feature modules.
- The nine-provider root graph, built in `app_wiring.dart`, is the single source of truth for how the app is wired.
- The repository layer's mock/real seam is the whole point of the frontend freeze: Sprint 2 swaps internals without touching UI code.

### References

- *FRONTEND\_ARCHITECTURE* (via *Mecha Connect Master Project Knowledge Base*, §3)
- *diagrams/frontend\_architecture.mmd, provider\_graph.mmd, navigation\_shell.mmd, repository\_layer.mmd, folder\_structure.mmd*

### Related Chapters

[Chapter 11 — Feature Modules Deep Dive](#) · [Chapter 12 — Navigation & App Shell](#) · [Chapter 15 — API Contract & Integration](#)

## Chapter 11 — Feature Modules Deep Dive

*Seven modules, one shared architecture. This chapter walks each module's purpose, inventory, key behaviour, and failure handling in turn.*

| Module        | Tab / entry      | Screens | Models         | Widgets | Tests |
|---------------|------------------|---------|----------------|---------|-------|
| AI            | Tab 3            | 5       | 7              | 10      | 25    |
| Auth          | Pre-shell        | 3       | 0              | 1       | 0*    |
| Home          | Tab 0 + Services | 2       | 1 (9 classes)  | 15      | 3     |
| Marketplace   | Services tab     | 8       | 8 (13 classes) | 19      | 43    |
| Mechanic      | Services tab     | 11      | 3 (6 classes)  | 8       | 10    |
| Fuel Delivery | Services tab     | 8       | 11             | 13      | 37    |
| Profile       | Tab 4            | 12      | 9 (13 classes) | 13      | 30    |

\* Auth shows zero dedicated module tests in the project data; its coverage is described as living inside the general widget and integration test suite rather than a module-specific file. This gap is discussed further in Chapter 16 and Chapter 20.

### 11.1 AI Assistant

The AI module is chat plus guided diagnosis plus conversation history, and it is also the app's cross-module dispatcher — action buttons inside AI responses route straight into Mechanic, Fuel, or Marketplace. At RC1 it runs a mock keyword-matching engine rather than a generative model.

- Models: AiBlock, AiActionButton, AiResponse, ChatMessage, Conversation, Diagnosis, QuickAction, SuggestedQuestion.
- Provider: AiProvider, the sole owner of one AiRepository instance shared with AiService and DiagnosisService (see Chapter 10.3).
- Seed data: five conversations (ai-0001 "Engine overheating" through ai-0005 "Oil change"), two pinned.
- Response blocks are typed — text, warning, recommendation, bulletList, checklist, or costEstimate — rather than free-form strings.
- Pull-to-refresh merges server data with local state via a `_mergeReloaded()` routine that preserves user conversations and pin overrides rather than overwriting them.

#### WARNING — Confidence Threshold

A diagnosis with confidence below 60% triggers a misdiagnosis disclaimer and a human-review prompt — a product-level mitigation for risk R4 (AI misdiagnosis), covered in Chapter 20.



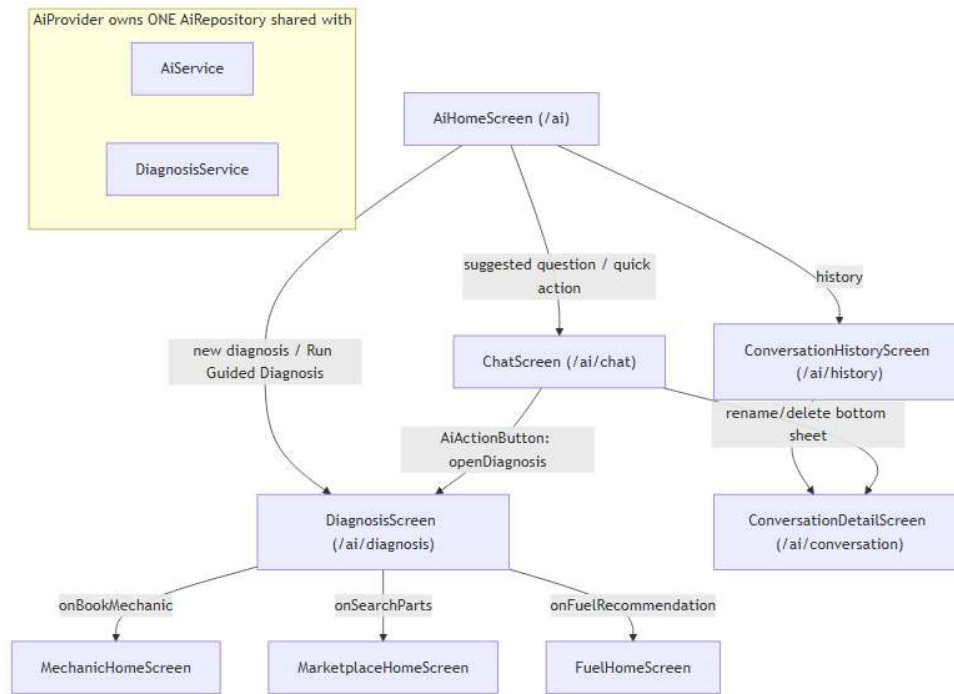


Figure 8. AI diagnosis and assistant flow, including the cross-module action buttons.

## 11.2 Auth

Login, sign-up, and forgot-password gate everything else. At RC1, session state is entirely local — a SharedPreferences flag, no credential storage — with Google Sign-In specified but not yet wired.

- Validation: email format, a minimum six-character password, and confirm-password matching.
- No models exist in this module — a legacy user.dart file was deliberately removed as dead code in an earlier sprint.
- Successful auth pushes the five-tab shell; logout is the only reverse path, clearing the navigation stack back to Login.

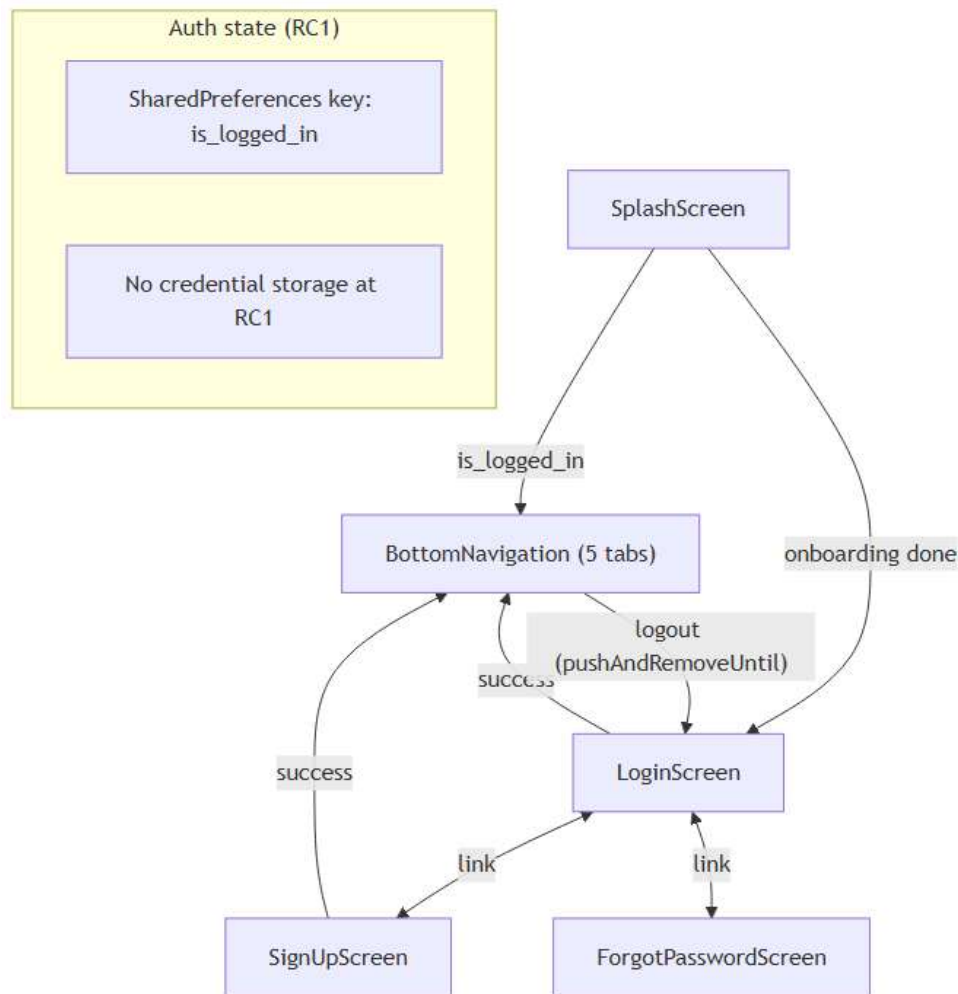


Figure 9. Auth and session flow, including the splash screen's three-way routing decision.

### 11.3 Home

Home aggregates everything a returning user needs on one dashboard: quick services, nearby services, marketplace highlights, recent activity, and current offers, plus the Services tab's card grid that fans out to the other three service modules.

- A single model file backs nine classes: the HomeData aggregate plus UserProfile, LocationInfo, VehicleInfo, QuickService, NearbyService, MarketplaceItem, ActivityItem, and OfferInfo.
- Service cards route out to VehicleFormPage (mechanic), FuelHomeScreen, MarketplaceHomeScreen, and HomeSearchScreen.
- Accessibility: the search bar and the SOS card are both wrapped in explicit Semantics.

### 11.4 Marketplace

A forty-product spare-parts catalogue with cart, wishlist, checkout, and order confirmation — and the module responsible for writing into the shared Orders feed.

- Seed catalogue: 40 products (including one deliberately out-of-stock item), 10 categories, 15 brands, 3 offers, 3 coupons.
- Product flags: featured, best-seller, trending, flash-deal, recommended.

- `placeOrder` both returns a typed `MarketplaceOrder` list (`MKP-<year>-<0000>`) and calls `addMarketplaceOrder` plus `orderStore.notify()` to write the Orders tab.
- `ProductCard` uses `context.read` for one-off reads and `context.select` for the wishlist flag specifically, so a wishlist toggle doesn't force a full card rebuild.
- Touch targets for wishlist and cart icons are held to at least 44px with tooltips, an explicit accessibility commitment.

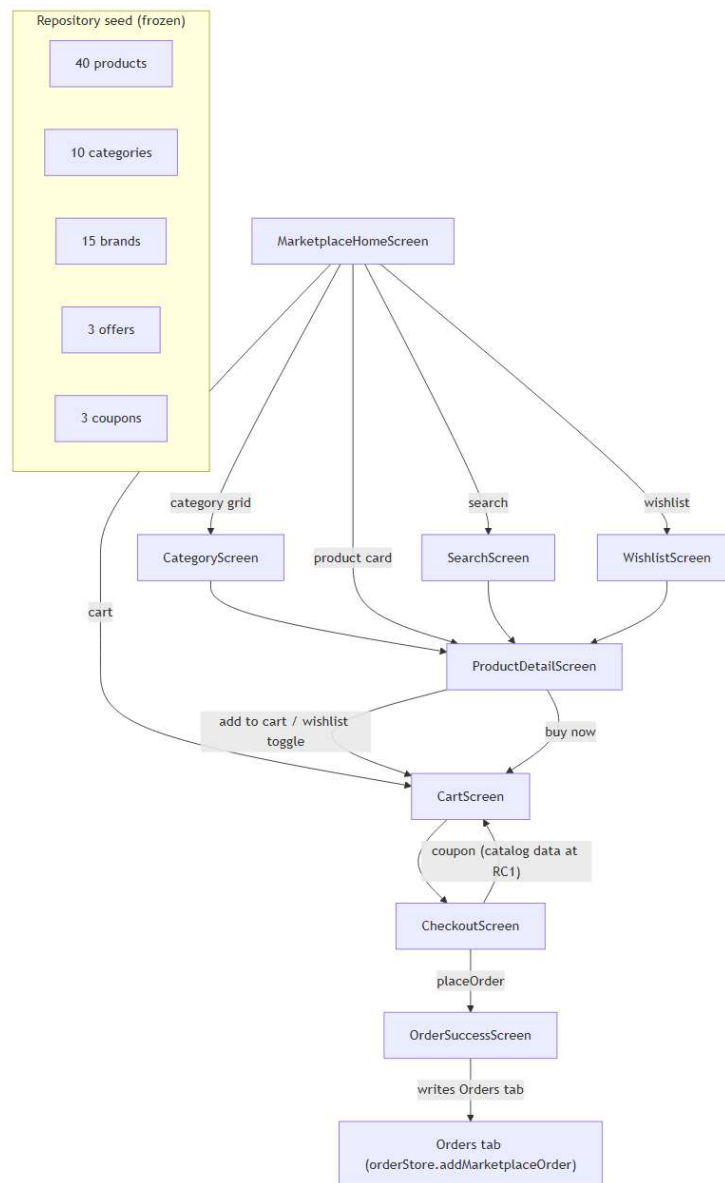


Figure 10. Marketplace browsing, cart, and checkout — including the write into the shared Orders feed.

## 11.5 Mechanic

The most screen-dense module in the app: browse, view details, select a service, optionally run AI diagnosis, book, track live, complete, rate, and review.

- Seed data: four mechanics (m1–m4, with m4 deliberately marked unavailable), three featured, reviews r1 through r8, eight categories, eight services.

- VehicleFormScreen calls the AI module's DiagnosisService directly — a mock call, no HTTP — since a fix in an earlier sprint removed a legacy real-HTTP call to a local address that could hang for up to 90 seconds.
- Live tracking runs a three-second progress timer isolated from the map widget, so the map itself never rebuilds on every tick.
- An availability surge of ₹100 is shown as an explicit, itemised cost line rather than folded silently into the total.

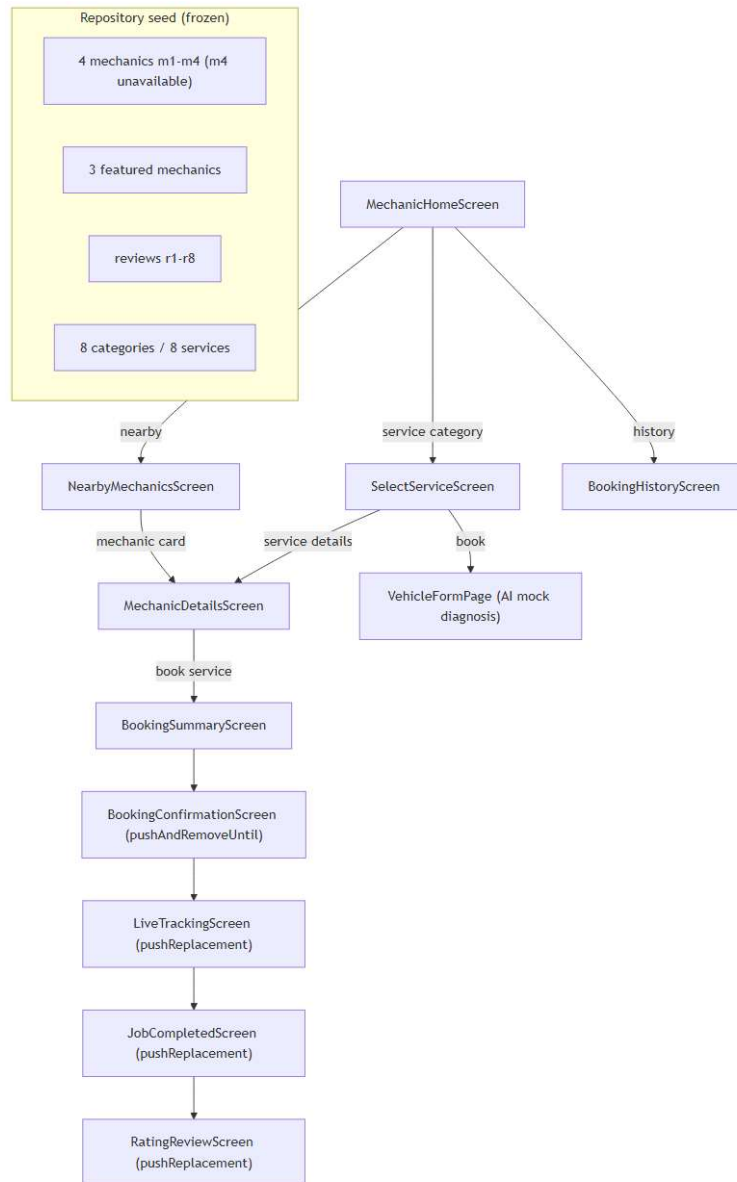


Figure 11. The full mechanic booking lifecycle, from browsing through rating and review.

## 11.6 Fuel Delivery

On-demand fuel delivery, and the module most shaped by real-world location constraints.

- Eleven models: DeliveryLocation, FuelOrder, FuelPartner, FuelStation, FuelVehicle, Invoice, PriceEstimate, TrackingInfo, plus FuelType, OrderStatus, and FuelConstants.

- GPS acquisition is capped at a 10-second settings limit and a 12-second hard timeout, with a manual-entry fallback — the fix for a blank-screen bug found in an earlier sprint.
- Status sequence: requested → accepted → fuelPacked → partnerAssigned → enRoute → arrived → delivered, with cancelled as a side branch at any point before delivery.
- Price formula: grand total equals fuel cost plus delivery charge plus platform fee plus taxes.
- Seed data: six stations, three saved vehicles, order history FUEL-2026-0005 through 0009.



Figure 12. Fuel delivery lifecycle, including the GPS-timeout and manual-entry fallback.

## 11.7 Profile

The account centre — everything a user manages about themselves, their vehicles, and their money.

- Thirteen model classes across nine files: EmergencyContact, NotificationSettings, ProfileStats, Reward, RewardTierProgress, RewardsData, SavedAddress, UserProfile, ProfileVehicle, WalletTransaction, Coupon, PaymentMethod, WalletData.
- Seed identity: Jagadeesh Gowda, Pro tier; two vehicles (veh-101/102), two addresses (addr-101/102), a wallet balance of ₹1,200 with 2,450 reward points across 4 transactions, 12 services on record, and a referral code (GOWDA200).
- Vehicles and addresses always sort default-first — a frozen ordering rule applied consistently across the module.
- fetchOrders reads the same shared ordersList that the Orders tab renders, guaranteeing the two views never disagree.

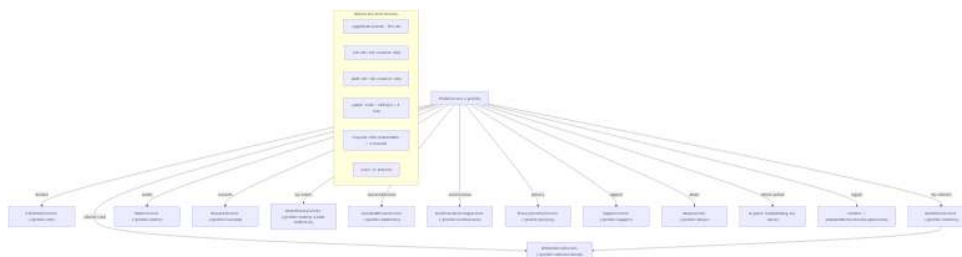


Figure 13. Profile and vehicle management, including the shared order-history read.

## 11.8 Cross-Module Dependency Map

No module in Mecha Connect is fully isolated. The dependency edges below are explicitly documented as frozen — changing any of them is an architectural decision, not a routine code change.

| From                            | To                            | Nature of the edge                   |
|---------------------------------|-------------------------------|--------------------------------------|
| Mechanic — VehicleFormScreen    | AI — DiagnosisService         | Direct mock call, no HTTP            |
| AI — action buttons             | Mechanic / Fuel / Marketplace | Deep-link navigation                 |
| Marketplace — placeOrder        | Orders tab (shared list)      | Writes via orderStore.notify()       |
| Profile — fetchOrders           | Orders tab (shared list)      | Reads the same list                  |
| Fuel / Mechanic — live tracking | LocationProvider              | Map centring, nearest-first ordering |

### Key Takeaways

- Seven modules share one architecture but each owns a distinct slice of the product: dispatch (AI), gatekeeping (Auth), aggregation (Home), commerce (Marketplace), services (Mechanic, Fuel), and account (Profile).
- Two shared seams — the AI diagnosis service and the Orders feed — are the connective tissue that make the app feel like one product instead of five bolted-together features.
- Auth's zero-test gap stands out precisely because every other module has module-specific coverage in the double digits.

### References

- *modules/\*.md (eight module files)*
- *modules/overview.md — Cross-Module Edges*
- *MASTER\_PROJECT\_DATA.json — modules*

### Related Chapters

[Chapter 10 — Frontend Architecture](#) · [Chapter 16 — Testing & QA](#) · [Chapter 20 — Risks & Mitigations](#)

## Chapter 12 — Navigation & App Shell

*How screens are addressed, how transitions feel, and where the seams between modules actually live in code.*

### 12.1 Routing Model

Mecha Connect declares exactly one named route — the root path, resolving to the splash screen. Everything downstream of that is imperative navigation: `MaterialPageRoute` pushes, with two custom fade transitions (`aiFadeRoute` at 220 milliseconds, `profileFadeRoute` at 220/180 milliseconds) reserved for the AI and Profile modules specifically.

### 12.2 Declared Route Constants

| Module      | Routes                                                                                                                                                      |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AI          | <code>/ai</code> , <code>/ai/chat</code> , <code>/ai/diagnosis</code> , <code>/ai/history</code> , <code>/ai/conversation</code>                            |
| Marketplace | <code>/marketplace</code> , <code>/marketplace/product</code> , <code>/category</code> , <code>/search</code> , <code>/cart</code> , <code>/wishlist</code> |
| Profile     | <code>/profile</code> + 11 sub-routes (edit, vehicles, vehicles/detail, addresses, wallet, rewards, orders, notifications, privacy, support, about)         |

Fuel Delivery, Mechanic, Auth, and Home have no dedicated `navigation.dart` file — they navigate inline via direct Navigator calls rather than declared route constants, which is a documented asymmetry in the codebase rather than an oversight to fix blindly (see Chapter 20 for how this is tracked).

### 12.3 Splash Decision Tree

- `is_logged_in` is true → push straight to the five-tab shell.
- `is_logged_in` is false, but onboarding has been completed → push to Login.
- Neither — first launch → push to Onboarding.

### 12.4 Cross-Module Edges (Frozen)

| From                 | To                               | Trigger                                                             |
|----------------------|----------------------------------|---------------------------------------------------------------------|
| Home / Services      | Mechanic form, Fuel, Marketplace | Service cards                                                       |
| AI chat / diagnosis  | Mechanic, Fuel, Marketplace      | AiAction buttons                                                    |
| Marketplace checkout | Orders tab                       | <code>addMarketplaceOrder</code> + <code>orderStore.notify()</code> |
| Profile              | Auth                             | Logout                                                              |
| Orders tab           | Services tab                     | "Explore Services" affordance                                       |

### 12.5 Transition Language

Splash to its destination: a 450-millisecond fade. AI: 220 milliseconds. Profile: 220/180 milliseconds depending on direction. Everything else uses the platform's default `MaterialPageRoute` transition, and tab switching itself uses `google_nav_bar`'s built-in 250-millisecond animation.

## Key Takeaways

- Only one route is declared by name; everything else is imperative navigation, deliberately.
- Three modules (AI, Marketplace, Profile) have dedicated route files; four do not, and that split is documented rather than accidental.
- Five cross-module navigation edges are explicitly frozen and form the connective tissue described in Chapter 11.8.

## References

- `NAVIGATION_MAP` (via `navigation/route_maps.md`)

## Related Chapters

[Chapter 10 — Frontend Architecture](#) · [Chapter 11 — Feature Modules Deep Dive](#)



## Chapter 13 — Design Tokens & Responsive System

*A closer look at how Chapter 8's design language survives across screen sizes, orientations, and accessibility settings.*

### 13.1 Token Recap

Chapter 8 introduced the full palette, spacing, radius, and typography scale. This chapter focuses on how those tokens behave under real device constraints rather than repeating the token table.

### 13.2 Breakpoints in Practice

| Breakpoint  | Threshold  | Effect                                      |
|-------------|------------|---------------------------------------------|
| Mobile      | < 600px    | Single-column layouts, full-width cards     |
| Tablet      | 600–1024px | Content width capped via ConstrainedContent |
| Content cap | > 1024px   | Maximum content width held at 480px         |

### 13.3 Motion & Accessibility

- Reduced-motion, an OS-level accessibility setting, disables the hero carousel's autoplay when active.
- Interactive controls throughout the app — not just on Home — carry explicit Semantics annotations for screen readers.
- DevicePreview, a debug-only device-frame wrapper, is active only when kDebugMode is true and never ships to production builds.

#### ENGINEERING NOTE — Why This Is a Separate Chapter From Chapter 8

Chapter 8 is the vocabulary — the tokens themselves. This chapter is the grammar — how those tokens combine under real constraints (small screens, reduced motion, tablets) rather than in isolation. Together they are meant to be read as one design-system story split for readability, not duplicated content.

### Key Takeaways

- Two breakpoints (600px, 1024px) and one content-width cap (480px) govern the entire app's responsive behaviour.
- Accessibility is handled as a first-class design concern — reduced motion and semantic labelling — not an afterthought.

### References

- *UI\_DESIGN\_SYSTEM* (via Mecha Connect Master Project Knowledge Base, §7)

### Related Chapters

[Chapter 8 — UI/UX Design System](#) · [Chapter 11 — Feature Modules Deep Dive](#)

## Chapter 14 — Data Layer & Database Blueprint

*Where the app's data lives today, and the PostgreSQL schema it's being built toward.*

### 14.1 Today: In-Memory Mock Reality

Every entity in the app currently lives in a feature module's models folder, backed by repository seed data — not a database of any kind. This is a deliberate RC1 decision, not an oversight: it lets the frontend be fully built, tested, and frozen before a single line of backend code has to be correct.

| Area                  | Seed facts                                                                       |
|-----------------------|----------------------------------------------------------------------------------|
| Marketplace catalogue | 40 products, 10 categories, 15 brands, 3 offers, 3 coupons                       |
| Mechanics             | 4 (m1–m4, one unavailable), 3 featured, reviews r1–r8, 8 categories, 8 services  |
| Fuel                  | 6 stations, 3 saved vehicles, history FUEL-2026-0005..0009                       |
| AI                    | 5 seed conversations, 2 pinned                                                   |
| Profile               | Jagadeesh Gowda (Pro), wallet ₹1,200 / 2,450 pts, 12 services, referral GOWDA200 |

### 14.2 Target Schema — PostgreSQL 15

The target schema mirrors the entity fields the UI already depends on, organised into six groups.

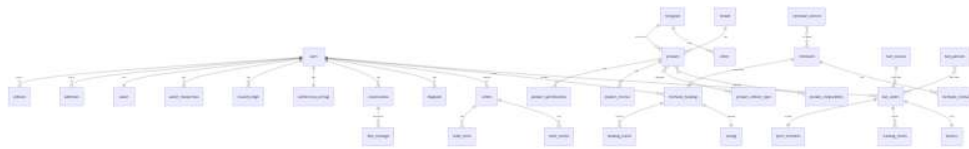


Figure 14. Entity-relationship diagram for the target PostgreSQL 15 schema.

| Group          | Core tables                                                                                                                                          |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Users & wallet | users, vehicles, addresses, wallet, wallet_transactions, reward_ledger, notification_settings                                                        |
| Marketplace    | categories, brands, products (+ specifications, vehicle types, compatibility, reviews), offers, coupons                                              |
| Orders         | orders (parent), order_items, order_entries — the unified Orders-tab feed                                                                            |
| Mechanic       | mechanics (+ skills, languages, working hours), mechanic_services, mechanic_categories, mechanic_reviews, mechanic_bookings, booking_events, ratings |
| Fuel           | fuel_orders, price_estimates, fuel_stations, fuel_partners, tracking_events, invoices                                                                |
| AI             | conversations, chat_messages, diagnoses                                                                                                              |

#### Conventions

- UUID primary keys throughout, TIMESTAMPTZ for timestamps.
- Money stored as NUMERIC(12,2) in Indian Rupees; percentages as NUMERIC(5,2).
- Status fields are VARCHAR with application-level enums enforced by CHECK constraints, matching the frozen client-side enums exactly.

- Soft deletes via a `deleted_at` column where the model calls for it; `created_at` and `updated_at` are present on every mutable table.

### 14.3 Client-to-Schema Mapping (Frozen)

A handful of mapping rules govern exactly how the mock model becomes the real one, and they are worth reading closely because they determine what changes and what doesn't when the backend goes live.

- The client's `ordersList` (a set of map entries) maps directly onto the `order_entries` table.
- `ProfileStats.orders` is computed as a count of `order_entries`; `ProfileStats.services` is a seed counter (12 in the current mock); `rewards` is a sum over `reward_ledger.points`.
- Vehicle and address counters start at 200 on the client; the seed data uses 101/102 for the two example vehicles and addresses.
- `is_logged_in` and `theme_mode` stay entirely on-device via `SharedPreferences` and are never written to the database — a deliberate boundary, not an omission.
- Live tracking in Sprint 2 will be backed by `tracking_events` plus a WebSocket push; the `TrackingInfo` payload shape itself is frozen and must not change.

#### **FROZEN (RC1) — Read This Table Before Writing Any Migration Code**

Every mapping rule above is stated as frozen in the source documentation. A Sprint 2 engineer who changes the shape of `TrackingInfo`, or moves `theme_mode` into the database, is not making a bug fix — they are breaking a documented contract that the frontend was built and tested against.

### Key Takeaways

- All RC1 data is in-memory; the PostgreSQL 15 schema below is target state, not current state.
- `order_entries` is the single most important table in the target schema — it's what the Orders tab and Profile order history both read from once real.
- Client-side fields that stay on-device (`is_logged_in`, `theme_mode`) are a deliberate, documented boundary — not gaps to be closed in Sprint 2.

### References

- *DATABASE\_BLUEPRINT* (via `database/data_model.md`, `database/schema.sql`)

### Related Chapters

[Chapter 11 — Feature Modules Deep Dive](#) · [Chapter 15 — API Contract & Integration](#)

## Chapter 15 — API Contract & Integration

*The frozen contract every repository already honours in mock form, and what the real FastAPI backend still has to build against it.*

### 15.1 Conventions

- Base path: /api/v1, the Sprint 2 target.
- Simulated latency by module: AI 900ms, Profile 800ms, Home 800ms, Marketplace 700ms, Fuel 700ms, Mechanic per-method.
- Failure injection: repositories accept a failForFirstCalls parameter and throw a typed XxxNetworkException, carrying a user-facing message, for a deterministic number of calls.
- Money is a double in Indian Rupees; timestamps are ISO-8601 on the wire.
- API evolution rule: additive field changes only. Any enum change requires a client-matching release — the client does not tolerate silent enum drift.

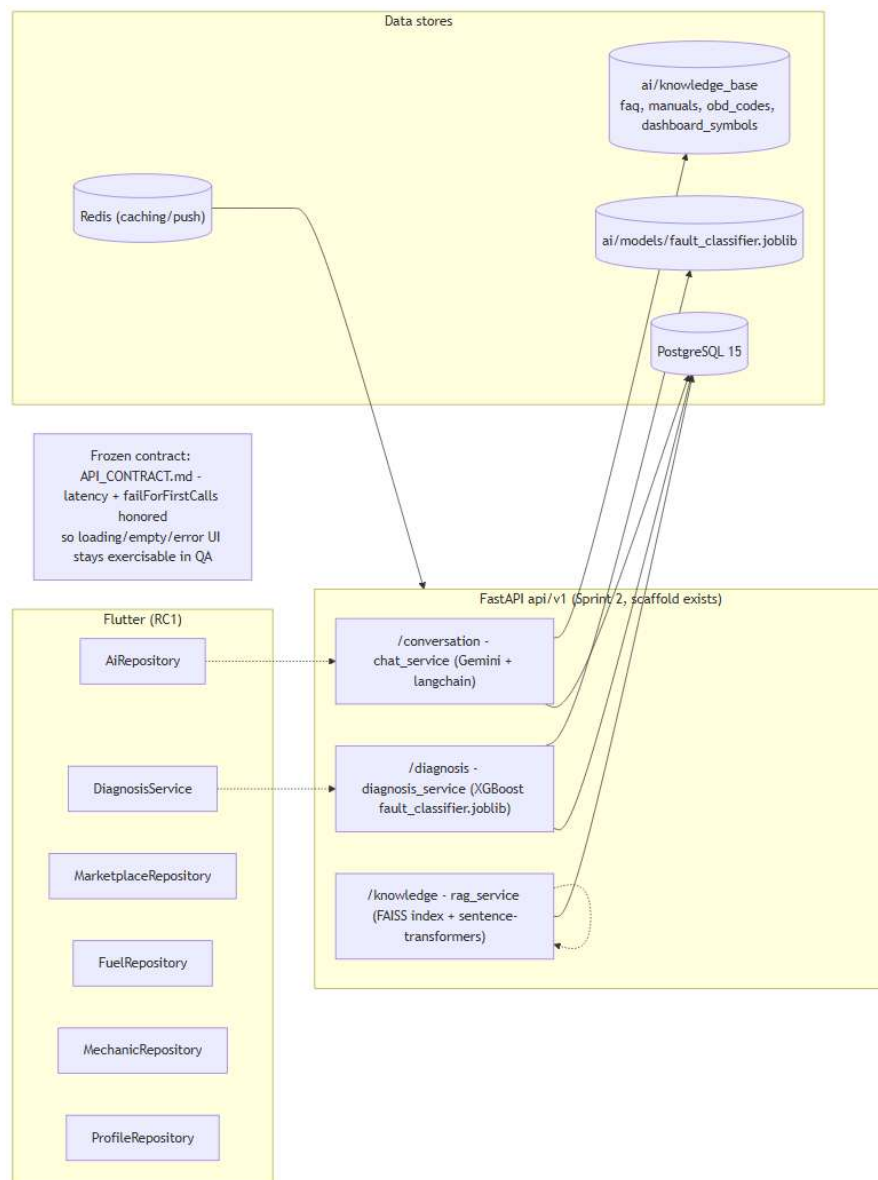


Figure 15. API contract flow — the shape every FastAPI endpoint must match once implemented.

## 15.2 Backend Scaffold vs. Full Contract

### WARNING — Only One of Five Modules Is Scaffolded

The FastAPI backend scaffold today mirrors only the AI module's surface — `conversation.py`, `diagnosis.py`, and `knowledge.py`, backed by a chat service (Gemini/langchain), a diagnosis service (an XGBoost fault-classifier model), and a retrieval service over a FAISS knowledge base (FAQs, manuals, OBD codes, dashboard symbols). Marketplace, Mechanic, Fuel, and Profile endpoints are explicitly not yet scaffolded, per the source API documentation.

## 15.3 Module Method Surfaces

| Module      | Representative methods                                                                                                                    |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Home        | <code>fetchHomeData()</code> → the full dashboard aggregate                                                                               |
| Auth        | Local-only at RC1; no backend methods yet                                                                                                 |
| AI          | <code>fetchConversations()</code> , <code>sendMessage()</code> , <code>diagnoseVehicle()</code>                                           |
| Marketplace | <code>fetchProducts/Categories/Brands/Offers</code> , <code>getCoupons()</code> , <code>createOrder()</code>                              |
| Mechanic    | List/query mechanics, fetch details and reviews, book, track, rate                                                                        |
| Fuel        | <code>getFuelStations()</code> , <code>createOrder()</code> , <code>acceptOrder()/advanceStatus()</code> , <code>generateInvoice()</code> |
| Profile     | <code>fetchProfile/Vehicles/Addresses/Wallet/Rewards/Stats/Orders</code> , plus the matching mutators                                     |

## 15.4 Versioning & Backward Compatibility

The one rule that governs every future API change: additive fields only. The client is written to ignore fields it doesn't recognise, which means new data can be added to a response at any time without a client release — but changing the meaning or values of an existing enum requires shipping a new client build in lockstep, because the frozen enums (order status, fuel status, etc.) are relied on directly in UI logic.

### Key Takeaways

- The API contract is fully specified for AI; the other four modules have a documented method surface but no backend implementation yet.
- `failForFirstCalls` and simulated latency aren't just testing conveniences — they're the contract the real backend has to feel like once it exists.
- Additive-only field changes are the one compatibility rule every future backend engineer needs to internalise before touching an endpoint.

### References

- `API_CONTRACT` (via `api/endpoint_catalog.md`)

### Related Chapters

[Chapter 10 — Frontend Architecture](#) · [Chapter 14 — Data Layer & Database Blueprint](#) · [Chapter 19 — Deployment & DevOps](#)

## Chapter 16 — Testing & QA

*162 tests, zero static-analysis issues, and one module that stands out for having neither.*

### 16.1 Test Coverage by Area

| Area                | Tests |
|---------------------|-------|
| AI                  | 25    |
| Fuel                | 37    |
| Marketplace         | 43    |
| Profile             | 30    |
| Mechanic            | 10    |
| Vehicle location    | 8     |
| Home dashboard      | 3     |
| Runtime integration | 2     |
| Widget              | 4     |
| Total               | 162   |

### 16.2 How Tests Are Structured

- Module tests drive real providers against mock repositories, including deliberately triggered failure paths via `failForFirstCalls` — so a passing test suite means the retry and error UI genuinely works, not just the happy path.
- The runtime integration test uses the exact production `buildRootProviders()` graph rather than a simplified test-only wiring, which means it catches provider-construction bugs (like the AI "triple-repo bug" mentioned in Chapter 10) that a narrower test setup would miss.
- flutter analyze reports zero issues, and flutter build web passes cleanly with no RenderFlex overflow errors and verified responsive behaviour.

#### WARNING — No Golden or Screenshot Tests at RC1

The project explicitly notes that no golden/screenshot tests exist at RC1 — the screenshot workspace referenced throughout this handbook's figures is forward-looking, not yet populated. Every screenshot placeholder in this handbook reflects that honestly rather than fabricating an image.

### 16.3 Auth's Test Gap

Auth is the one module in the inventory with zero tests attributed directly to it — its coverage is described only as living inside the general `widget_test.dart` and integration suite, without a dedicated module test file the way every other module has. Given that Auth gates every other flow in the app, and that Sprint 2 plans to replace its entire backend (SharedPreferences flag → Firebase Auth + JWT), this is worth closing before that migration begins rather than after.

### Key Takeaways

- 162 of 162 tests pass, spanning module logic, widget behaviour, vehicle location, and one full-graph integration test.
- Failure-injection testing (failForFirstCalls) means error and retry UI are exercised by the suite, not just assumed correct.
- Auth's lack of dedicated tests is the most concrete, checkable gap in an otherwise strong testing story.

## References

- *QA\_CERTIFICATION\_REPORT, FRONTEND\_LOCK\_REPORT (via Mecha Connect Master Project Knowledge Base, §8)*

## Related Chapters

[Chapter 11 — Feature Modules Deep Dive](#) · [Chapter 18 — Certification & Frontend Lock](#) · [Chapter 20 — Risks & Mitigations](#)

## Chapter 17 — Release History & Versioning

*The path from an empty repository to a locked-frontend release candidate.*

### 17.1 Version Timeline

| Version | Date       | Milestone                                                  |
|---------|------------|------------------------------------------------------------|
| 0.0.1   | 2026-07-20 | Project initialised                                        |
| 0.1.0   | —          | Sprints 1.1–1.3 (foundations)                              |
| 0.4.0   | —          | Sprint 1.6 — mechanic booking lifecycle                    |
| 1.2.0   | 2026-07-30 | Sprint 1.7A — fuel delivery, including the GPS-timeout fix |
| 1.9.0   | 2026-08-02 | Sprint 1.9B — RC1 certification pass                       |
| 1.9.1   | 2026-08-05 | Sprint 1.9b — close-out                                    |
| 1.9.2   | 2026-08-05 | Sprint 1.9b — final review                                 |
| 1.0.0+1 | 2026-08-05 | RC1 release — Frontend Lock Candidate                      |

Dates between 0.1.0 and 1.2.0 are not recorded in the source material and are left blank rather than estimated. The release tag v1.0.0-rc1 is documented as a required manual step that, per the source bundle, has not yet been created.

### 17.2 Versioning Policy

- Development builds: dev-{hash}
- Pre-release stages: alpha-{build}, beta-{build}, rc-{build}
- Production releases: {major}.{minor}.{patch}

### 17.3 Verification Baseline (Frozen)

- flutter analyze → 0 issues
- flutter test → 162/162 passing
- flutter build web → passes cleanly

#### Key Takeaways

- The app moved from initialisation to a Frontend Lock Candidate in roughly two and a half weeks of documented sprints.
- A four-tier versioning policy (dev/alpha/beta/rc → production semver) is already defined for future releases.
- The v1.0.0-rc1 git tag is a known outstanding manual step, not yet completed as of this handbook's compilation date.

#### References

- *VERSION\_HISTORY, PROJECT\_STATUS\_REPORT, RC1\_RELEASE\_REPORT (via metadata/version\_matrix.md)*

#### Related Chapters





## Chapter 18 — Certification & Frontend Lock

*What "Frontend Lock Candidate" actually means, and what it deliberately does not mean.*

### 18.1 The Terminology, Precisely

#### **FROZEN (RC1) — Certification Wording**

The project uses the phrase "Frontend Lock Candidate" and explicitly retires the phrase "RC1 Certified." This handbook follows that discipline throughout — the distinction matters because "Frontend Lock Candidate" describes a frozen, verified state pending approval, not a final, unconditional certification.

### 18.2 What Is Frozen

- The tab shell and navigation structure.
- Every design-system token — colour, spacing, radius, elevation, typography.
- All seven feature modules, as they exist at RC1.
- Data models and repository interfaces (the seam Sprint 2 will use, described in Chapter 10.5).

### 18.3 What Is Still Permitted

Within the frozen scope, five categories of change remain open: bug fixes, accessibility fixes, responsive-layout fixes, state-management fixes, and documentation updates. Any such change must still clear the verification gate described in Chapter 17.3 — zero analyzer issues, all 162 tests passing — before it's considered complete.

### 18.4 Verification Gate

| Check                             | Result                                              |
|-----------------------------------|-----------------------------------------------------|
| Static analysis (flutter analyze) | 0 issues                                            |
| Automated tests (flutter test)    | 162 / 162 passing                                   |
| Web build (flutter build web)     | Passes; no RenderFlex overflow; responsive verified |

#### Key Takeaways

- "Frontend Lock Candidate" is a precise, deliberately conditional status — not a synonym for "finished."
- The freeze covers shell, tokens, modules, and repository interfaces, while still permitting bug, accessibility, responsive, and state fixes.
- Every permitted change is gated by the same zero-issues, 162-tests-passing bar that produced the RC1 build in the first place.

#### References

- [\*FRONTEND\\_LOCK\\_REPORT\*](#) (via Mecha Connect Master Project Knowledge Base, §3.7, §9)

#### Related Chapters

[Chapter 10 — Frontend Architecture](#) · [Chapter 16 — Testing & QA](#) · [Chapter 17 — Release History & Versioning](#)

## Chapter 19 — Deployment & DevOps

*A placeholder chapter, deliberately left incomplete rather than fabricated.*

### FUTURE WORK — NOT AVAILABLE YET

No hosting decision, environment strategy (staging vs. production), CI/CD pipeline description, or secrets-management approach appears anywhere in the reviewed documentation bundle — not even as a stub or a stated TBD.

This is a genuine gap, not a documentation oversight to paper over: Sprint 2's plan to deploy FastAPI, migrate to PostgreSQL, and integrate Firebase Auth has no accompanying deployment plan yet.

### 19.1 What Is Known

- The backend targets FastAPI, PostgreSQL 15, and Redis, running against Python 3.13 in the current scaffold environment (see Chapter 9.2).
- Firebase Auth + JWT is the named authentication target (Chapter 9.2, Chapter 11.2).
- Beyond those technology choices, no operational detail — hosting provider, containerisation, environment promotion, monitoring, or rollback strategy — is documented.

### 19.2 What Would Complete This Chapter

- A named hosting/infrastructure decision and environment topology (dev / staging / production).
- A CI/CD pipeline description — build, test, and deployment automation for both the Flutter client and the FastAPI backend.
- A secrets-management approach, particularly relevant given flutter\_dotenv is already a client dependency (Chapter 9.1) but its production secret-handling strategy is not documented.
- Monitoring, logging, and incident-response expectations for the live backend once it exists.

### Key Takeaways

- This chapter is intentionally incomplete: no deployment or DevOps source material exists in the bundle reviewed.
- Deciding these questions before Sprint 2 begins, rather than during it, is one of this handbook's standing recommendations (see the companion engineering review).

### References

- *No source document identified in the delivered bundle*

### Related Chapters

[Chapter 9 — Technology Stack](#) · [Chapter 15 — API Contract & Integration](#) · [Chapter 21 — Future Scope & Next Steps](#)

## Chapter 20 — Risks & Mitigations

*Five named risks and two named debts, each with a stated mitigation or resolution path.*

### 20.1 Risk Register

| ID | Risk                        | Severity / Likelihood | Mitigation                                              |
|----|-----------------------------|-----------------------|---------------------------------------------------------|
| R1 | GPS denied or disabled      | High / High           | Graceful state machine with a manual-entry fallback     |
| R2 | Network failure mid-booking | —                     | Offline queue with retry                                |
| R3 | Mechanic no-show            | —                     | Penalty, auto-reassignment, and user notification       |
| R4 | AI misdiagnosis             | —                     | Confidence-below-60% disclaimer and human-review prompt |
| R9 | Location permission denied  | —                     | Explicit explanation plus manual entry                  |

Risk identifiers R5 through R8 are referenced by the numbering gap above but their content is not present in the delivered documentation bundle; they are not reproduced here rather than guessed at.

### 20.2 Technical Debt Register

| ID  | Debt                                                                                                                             | Target resolution          |
|-----|----------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| TD1 | Home and Services screens live under the legacy lib/starting_screen/ path instead of a feature-first lib/features/home/ location | Not yet scheduled          |
| TD3 | Mock data is hardcoded across every repository                                                                                   | Sprint 2 (repository swap) |
| TD4 | No backend error handling exists in the UI layer yet, because there is no real backend to fail against                           | Sprint 2                   |

### 20.3 Reading the Register Honestly

Four of the five documented risks (R2, R3, R4, R9) are product-level mitigations that assume backend and business-process support (offline queues, reassignment logic, human review workflows) that doesn't exist yet in the mock-only frontend — they describe intended behaviour, not verified behaviour. R1 stands apart because its mitigation (the GPS timeout and manual-entry fallback) is already implemented and tested in the Fuel Delivery module today, as described in Chapter 11.6.

#### Key Takeaways

- R1 (GPS denial) is the one risk with an implemented, tested mitigation already in the frontend; the rest describe intended Sprint 2+ behaviour.
- TD1's legacy folder naming is the most visible, longest-standing piece of technical debt and has no assigned resolution sprint.
- TD3 and TD4 are really the same underlying gap — no real backend exists yet — described from two different angles (data, error handling).

## References

- *RISK\_ANALYSIS (via Mecha Connect Master Project Knowledge Base, §10)*

## Related Chapters

[Chapter 7 — User Experience & Journeys](#) · [Chapter 11 — Feature Modules Deep Dive](#) · [Chapter 21 — Future Scope & Next Steps](#)

## Chapter 21 — Future Scope & Next Steps

*Where the roadmap points beyond Sprint 2, in as much detail as the source material provides.*

### 21.1 The Near-Term Roadmap

| Sprint   | Focus                                                                                        |
|----------|----------------------------------------------------------------------------------------------|
| Sprint 2 | FastAPI deployment, PostgreSQL migration, full API contract, Firebase Auth, Maps integration |
| Sprint 3 | Production polish; B2B AI-diagnosis API monetisation (₹999/fleet/month) goes live            |
| Sprint 4 | A partner-facing app, for mechanics and fuel partners                                        |
| Sprint 5 | An admin dashboard                                                                           |

### 21.2 What's Genuinely Ready to Build On

- The repository seam (Chapter 10.5) means Sprint 2 is additive to the frontend, not disruptive — a real strength worth carrying into any investor conversation.
- The AI backend scaffold (Chapter 15.2) is a running start on the highest-differentiation feature in the product (Chapter 1.3).
- The unit economics in Chapter 3, while unaudited, are strong enough on paper to support the Sprint 3 B2B monetisation plan if they hold up under real usage.

### 21.3 What Needs a Decision Before It Can Be Built On

- Deployment and DevOps strategy (Chapter 19) — undecided, and a prerequisite for Sprint 2's stated backend work.
- Competitive positioning (Chapter 5) — undocumented, and relevant before Sprint 3's monetisation push goes to market.
- Auth test coverage (Chapter 16.3) — the one clear, fixable gap standing between today and a backend migration of the app's most sensitive module.

#### TIP — Reading This Chapter Alongside the Companion Engineering Review

This handbook describes what the bundle documents. A separate pre-Sprint-2 engineering review (produced from the same source bundle) goes further and prioritises these gaps as P0/P1/P2 action items with concrete next steps — the two documents are meant to be read together.

### Key Takeaways

- The roadmap runs Sprint 2 (backend) → Sprint 3 (polish + monetisation) → Sprint 4 (partner app) → Sprint 5 (admin dashboard).
- The frontend's architecture is genuinely ready to support this roadmap; deployment strategy and competitive positioning are the two clearest gaps standing in front of it.

### References

- *Mecha Connect Master Project Knowledge Base, §9*
- `MASTER_PROJECT_DATA.json`

## Related Chapters

[Chapter 3 — Business Model & Unit Economics](#) · [Chapter 6 — Feature Overview & Roadmap](#) · [Chapter 19 — Deployment & DevOps](#)



## Appendix A — Glossary

*Terms used throughout this handbook, as defined in the source documentation.*

| Term                     | Definition                                                                                |
|--------------------------|-------------------------------------------------------------------------------------------|
| RC1                      | Release Candidate 1 — the current deliverable state, version 1.0.0+1                      |
| Frontend Lock Candidate  | Frozen-frontend status; UI, data, and navigation frozen pending RC1 approval              |
| Repository seam          | The interface between providers and mock/real data; the only data source in the app       |
| Mock realism             | Simulated latency plus failForFirstCalls deterministic failure injection                  |
| failForFirstCalls        | A mock failure-injection flag: the first N calls throw typed exceptions                   |
| OrderStore / ordersList  | Shared singletons in lib/parts/order_data.dart; the unified Orders-tab feed               |
| order_entries            | The PostgreSQL table ordersList maps onto in Sprint 2                                     |
| AiBlock / AiResponse     | Typed assistant reply blocks and their associated action buttons                          |
| AiAction                 | openDiagnosis, bookMechanic, searchParts, or fuelRecommendation                           |
| IndexedStack             | The shell widget that keeps all five tabs mounted simultaneously                          |
| GNav                     | The google_nav_bar bottom-navigation widget                                               |
| ETA                      | Estimated time of arrival, used for mechanic and fuel tracking                            |
| PUC                      | Pollution Under Control certificate, tracked per vehicle                                  |
| AOV / LTV / CAC          | Average order value / lifetime value / customer acquisition cost                          |
| health_score             | A vehicle's health rating, 0–100                                                          |
| membership_tier          | free or pro, a Profile-level attribute                                                    |
| OrderStatus (fuel)       | requested → accepted → fuelPacked → partnerAssigned → enRoute → arrived → delivered       |
| Reduced motion           | An OS accessibility setting that disables hero-carousel autoplay when active              |
| DevicePreview            | A debug-only device-frame wrapper, active only when kDebugMode is true                    |
| SharedPreferences        | The on-device key-value store backing is_logged_in, theme_mode, and notification settings |
| FAISS / RAG              | The backend AI index and retrieval-augmented-generation approach (Sprint 2 scaffold)      |
| XGBoost fault_classifier | The trained fault-diagnosis model behind guided diagnosis (Sprint 2 scaffold)             |

## Appendix B — ID Scheme Quick Reference

*Every frozen identifier prefix used across the app, in one place.*

| Domain                                | Prefix                          | Example                     |
|---------------------------------------|---------------------------------|-----------------------------|
| Marketplace order                     | MKP-<year>-<0000>               | MKP-2026-0001               |
| Marketplace product                   | p-                              | p-spark-plug                |
| Coupon / offer                        | coupon- / offer-                | coupon-mecha10, offer-brake |
| Review (product/mechanic)             | rv- / r                         | rv-brake-1, r1              |
| Shared orders feed                    | ORD-                            | ORD-1001                    |
| Fuel order                            | FUEL-<year>-<0000>              | FUEL-2026-0009              |
| Fuel station / partner                | station_ / partner_             | station_1, partner_1        |
| Fuel invoice                          | INV-<orderId>                   | INV-FUEL-2026-0009          |
| Mechanic service / mechanic           | svc_ / m                        | svc_1, m1                   |
| AI conversation / message / diagnosis | ai- / m- / diag-                | ai-0001, m-0001, diag-1     |
| Profile vehicle / address             | veh- / addr- (counter from 200) | veh-101, addr-202           |
| Wallet transaction / reward / payment | txn- / rew- / pay-              | txn-101, rew-101, pay-101   |

## Appendix C — Figure Index

*Every diagram embedded in this handbook, with its source and the chapter it illustrates.*

| Fig. | Title                             | Chapter |
|------|-----------------------------------|---------|
| 1    | System architecture               | 1       |
| 2    | Frontend architecture & entry     | 10      |
| 3    | System architecture (full)        | 10      |
| 4    | Provider graph                    | 10      |
| 5    | Navigation shell & five tabs      | 10      |
| 6    | Repository layer (mock → backend) | 10      |
| 7    | Folder structure                  | 10      |
| 8    | AI flows                          | 11      |
| 9    | Auth flow                         | 11      |
| 10   | Marketplace flows                 | 11      |
| 11   | Mechanic flows                    | 11      |
| 12   | Fuel flows                        | 11      |
| 13   | Profile flows                     | 11      |
| 14   | ER diagram (PostgreSQL)           | 14      |
| 15   | API flow                          | 15      |

### **FUTURE WORK — NOT AVAILABLE YET — Screenshots**

This handbook embeds all fifteen architecture and workflow diagrams from the source bundle. No product screenshots are embedded, because the source documentation explicitly states that no golden or screenshot tests exist yet at RC1 — the screenshot capture plan covering 39 additional figures across every module is itself Future Work, tracked in the bundle's own screenshot manifest.

## Appendix D — Source Document Index

---

*Canonical sources cited throughout this handbook, for traceability.*

- MASTER\_PROJECT\_KNOWLEDGE\_BASE.md — merged project knowledge
- MASTER\_PROJECT\_DATA.json — machine-readable project facts
- modules/\*.md — eight per-module knowledge files
- workflows/\*.md — seven workflow files with narrative, Mermaid, and failure/recovery detail
- navigation/route\_maps.md, api/endpoint\_catalog.md, database/data\_model.md, database/schema.sql
- manifests/figures\_manifest.md, diagrams\_manifest.md, tables\_manifest.md, screenshot\_manifest.md, asset\_manifest.md
- metadata/glossary.md, cross\_references.md, version\_matrix.md
- diagrams/\*.mmd, diagrams/svg/\*.svg, diagrams/png/\*.png — fifteen diagrams in three formats each

This handbook draws exclusively on the files listed above. Where a chapter's underlying source document (for example, the product requirements document referenced for competitive analysis, or any deployment/DevOps documentation) was not present in the delivered bundle, that chapter says so explicitly rather than filling the gap from general knowledge.